Supplemental Material: Formal Analysis and Proofs

Anonymized. Accompanies "Short Paper: Barry: A Streaming Storage Engine for Temporal Graphs".

```
# Barry: Formal Analysis

Complexity table, theorems, and proofs backing the paper's algorithmic claims.
This file ships with the artifact (the TVA precedent: proofs live in the repo,
the paper cites them).

Notation: `E` edges ingested, `V` vertices, `S` time span, `s` the shift,
`W = ceil(S / 2^s)` timeline windows, `d(v)` out-degree of `v`, `R` a query's
result set, `B` block capacity.

---

## 1  Complexity of core operations

| Operation | Barry (compacted) | Barry (uncompacted) | time-sorted CSR | entity CSR (arrival order) |
|---|---|---|---|---|
| append one edge | **O(1) amortized** | **O(1) amortized** | O(E) (rebuild) | O(1) amortized |
| point query `query_at(t)` | O(mu + \|R\|), mu = window occupancy, < 4*target by Thm 3 | same | O(V log d)
| O(E) scan |
| range `query_range(lo,hi)` | O(W_range + mu + \|R\|), boundary windows filtered, interior bulk-copied |
same | O(V log d + \|R\|) | O(E) scan |
| neighbor expansion in `[t_u, hi]` | O(log d(v) + k) | O(d(v)) | O(log d(v) + k) | O(d(v)) |
| forward/backward taint | O(Σ log d + \|R\| log \|R\| + k_total) | O(Σ d + ...) | same as compacted | O(Σ d
+ ...) |
| retention `evict_before(t)` | O(W_evicted + blocks) | same | O(E) rebuild | O(E) delete+compact |
| compaction `finalize()` | O(E log B_max) one-off, optional | — | O(E log E), **mandatory** | — |

Two rows carry the paper: **append** (the static structures' O(E) rebuild is E12's
33.2% wrong-answer window) and **neighbor expansion** (the log-vs-linear gap that
E13 measures directly at the storage layer).

---

## 2  Theorems

### Theorem 1 (Constant-time temporal addressing)
*For any timestamp `t` in `[T_start, T_start + 2^64)`, the timeline slot holding
every edge with timestamp `t` is computed in O(1) worst case, with no data-dependent
memory access.*

**Proof.** The map is `idx = (t − T_start) >> s`: one subtraction, one shift.
Injectivity onto windows: `t` and `t'` map to the same slot iff
`⌊(t−T_start)/2^s⌋ = ⌊(t'−T_start)/2^s⌋`, i.e. iff they share a window — so every
edge with timestamp `t` is registered under exactly `idx`, and no other slot can
hold one. No search structure is consulted; the bound is unconditional, not
expected-case. ■

Contrast: a B-tree/LSM index gives O(log E) with data-dependent pointer chases; a
hash index gives expected O(1) with worst-case collisions. The mapping is the only
scheme of the three whose *worst case* is a two-instruction computation.

### Theorem 2 (Physical-space bound under demand paging)
*Let `P` be the page size and `W_occ ≤ min(E, W)` the number of non-empty windows.
Physical memory consumed by the timeline array is at most
`W_occ · P + O(W · sizeof(ptr) / P)` pages-worth, independent of the virtual
reservation.*

**Proof.** The timeline is an anonymous mapping of `W` pointer slots. A page of it
becomes resident only when first written; a slot is written only when its window
receives its first edge (CAS install of the block pointer). Hence resident
timeline pages ≤ pages containing ≥1 occupied slot ≤ `W_occ` (each occupied slot
dirties at most one page) plus the fixed page-table overhead of the reservation,
which the kernel materializes lazily at `O(sizeof(ptr)·W/P)` page-table entries
only for touched ranges. Untouched windows cost 0 bytes of RAM. ■

This is the "zero-index" claim in its defensible form: sparsity is absorbed by the
MMU, not by a data structure Barry maintains.

### Theorem 3 (Adaptive shift bound — Algorithm 2)
*For `E ≥ 2·target` and non-degenerate span, `auto_select_shift_from_stats(E, t_min,
t_max, target)` returns `s` such that the mean occupancy `μ = E / W`, with
```

`W = ⌊S / 2^s⌋` non-degenerate windows, satisfies `target/2 ≤ μ < 4·target`.*

Proof. The implementation computes `D = ⌊E/target⌋` desired windows, a desired window size `w = ⌊S/D⌋`, and returns the smallest `s` with `2^s ≥ w`, i.e. `2^s ∈ [w, 2w)`. Then `W = ⌊S/2^s⌋` obeys `W > S/(2w) ≥ D/2` and `W ≤ S/w + 1 ≤ D + 1`. Hence `μ = E/W ≥ E/(D+1) ≥ E/(E/target + 1) = target·E/(E+target) ≥ target/2` for `E ≥ target`, and `μ < E/(D/2) = 2E/⌊E/target⌋ ≤ 2·target·E/(E−target) < 4·target` for `E ≥ 2·target`. ■

The two factors of two are honest artifacts of the design: one from rounding the window size up to a power of two (shift addressing admits nothing finer), one from the ceiling on the window count. A counterexample shows the bound is not `2·target`: `E = 190, target = 64` yields `D = 2` and can land all edges in a single window, `μ = 190 > 128`. The paper must quote `[target/2, 4·target)`; quoting a tighter bound would misstate the code.

Theorem 4 (Sparse taint correctness)
`forward_taint(v⋆, t⋆, t_hi)` returns exactly the set `{(v, a(v))}` where `a(v)` is the minimum, over time-respecting paths `v⋆ = u⋆ →^{e⋆} u⋆ → … →^{e_k} u_k = v` with `t⋆ ≤ ts(e⋆) ≤ … ≤ ts(e_k) ≤ t_hi`, of the arrival time `ts(e_k)` — i.e. the earliest possible contamination time — and omits every vertex with no such path.

Proof. Induction on extraction order from the min-heap, mirroring Dijkstra's invariant. Key facts: (i) arrival times along a time-respecting path are non-decreasing, so path prefixes are themselves optimal-substructure candidates; (ii) when `(t_u, u)` is popped with `t_u` equal to its `first_seen[u]`, no future relaxation can lower `a(u)`, because every heap entry has key ≥ `t_u` and edge timestamps only extend arrival times forward; (iii) each relaxation inserts `w` with the timestamp of a real edge from `u` at time ≥ `a(u)`, so every recorded arrival is witnessed by an actual path. Stale heap entries are discarded by the `first_seen` check. Termination: each vertex is expanded at most once per improvement, improvements are strictly decreasing and bounded. The dense `earliest_arrival` computes the same fixpoint over a dense array; `test_sparse_matches_dense_earliest_arrival` verifies vertex-for-vertex equality mechanically. Backward tracing is the mirror image under time reversal (max-heap, latest-influence times). ■

Theorem 5 (Bounded-step append)
In the absence of arena exhaustion, `insert` completes in a number of steps bounded by O(1 + c), where `c` is the number of CAS conflicts encountered, and system-wide at least one insert makes progress at every step (lock-freedom).

Proof. The write path is: locate slot (Theorem 1, O(1)); if empty, allocate block (one `fetch_add` on the arena bump — wait-free) and CAS-install; on CAS failure another insert installed a block, so proceed with the winner (progress transferred, not lost). Claiming a slot inside a block is a single `fetch_add` returning a unique index — wait-free. Overflow chains prepend by CAS with the same argument: a failed CAS implies a competing prepend succeeded. Every failure mode of one thread is a completed step of another; no thread ever waits on a lock or retries without global progress. Registered-index appends (`append_ref`) follow the identical pattern. ■

Note the honest scope: individual appends are lock-free, not wait-free — a thread can in principle lose every CAS. Wait-freedom holds for the slot-claim and arena-bump sub-steps only. The paper says "lock-free ingestion, wait-free slot claim", not more.

Remark (publication protocol, concurrent reads). Queries are correct during concurrent ingestion by a per-slot release-publication discipline:

- *Arena edges*: payload (`src`,`dst`,`weight`) is written first; `timestamp` is stored last with release. Fresh arena pages are zeroed, so `ts == 0` identifies an unpublished slot; every reader acquire-loads the timestamp and skips zeros. Consequently `ts == 0` is reserved: `insert` clamps an epoch-zero event to 1 (one time unit of error at the epoch boundary, never a dropped record).
- *Vertex-index refs*: `dst`/`offset` first; the `block` pointer last with release. `block == nullptr` marks an unpublished ref; readers skip.
- *Sorted-flag ordering*: an append clears `kBucketSorted` (release) **before** bumping the bucket count (`fetch_add`, acq-rel). A reader loads the count (acquire) and then the flag. If it observes the flag still set, then no append's clear — and hence, by program order, no append's bump — is visible to it; the acquire on the count synchronizes with every visible bump's release, so the loaded count covers only the compaction-verified prefix and binary

  search over it is sound. Any reader that instead sees a published new ref also
  sees the cleared flag and takes the linear path.

Verified by a multi-writer/multi-reader stress test (4 writers, 3 readers,
range + taint queries throughout ingestion): zero violations, final state equal
to a serially built engine.

Boundary (SIMD path, x86-TSO assumption). The AVX2 range scan gathers
timestamps with plain vector loads and upgrades the mask-dependent payload reads
with an acquire fence. Correctness of skipping unpublished slots relies on
`t_lo ≥ 1` (a zero timestamp fails the range test) and, for payload visibility,
on the acquire fence pairing with the writer's release store — sufficient on
x86-TSO, where the gather cannot observe a timestamp before earlier payload
stores are visible after the fence. A port to weaker memory models (ARM) must
re-audit this path or fall back to the scalar acquire loop.

Theorem 6 (Recovery is re-ingestion)
*Let `L` be a WAL whose complete frames contain edge batches `B■, …, B_k` and
whose final bytes may be torn. Replaying the complete frames of `L` through the
normal insert path yields an engine whose answer to every query in Barry's
interface equals that of the pre-crash engine restricted to acknowledged
batches.*

Proof. Engine state is determined by the multiset of ingested edges: inserts
are the only mutation, they commute (Theorem 5's appends are order-insensitive up
to bucket ordering, and every query is defined over edge sets filtered by
predicates, never over physical positions -- compaction changes cost, not
answers). The WAL frame check (length + CRC) accepts exactly the acknowledged
batches: a torn or corrupt tail fails its check and is discarded, and every
earlier frame was durable before its append returned. Hence replay ingests
precisely the acknowledged multiset, and query-equivalence follows from state
being a function of that multiset. No undo, redo, or idempotence bookkeeping is
required. ■

---

3 What this analysis deliberately does not claim

- No claim that expansion beats a sorted static structure: Theorem-1 addressing
  finds the *window*, not the neighbor; per-vertex expansion is `O(log d + k)`
  only after compaction (§1), and tcsr achieves the same bound always.
- No claim of durability: the analysis covers the in-memory structure. WAL and
  time-prefix persistence are future work (see NEXT steps).
- Theorem 2 bounds the *timeline*; blocks and the vertex index consume memory
  proportional to E and are reported empirically (E5).